

Software Requirements As Negotiated Win Conditions

Barry Boehm, Prasanta Bose, Ellis Horowitz, and Ming-June Lee

USC Center for Software Engineering
Department of Computer Science
University of Southern California
Los Angeles, California

e-mail: (boehm, pbose, horowitz, milee)@pollux.usc.edu

Abstract

Current processes and support systems for software requirements determination and analysis often neglect critical needs of important classes of stakeholders and limit themselves to concerns of the developers, users and customers. Besides developers, customers, and users, these stakeholders can include maintainers, interfacers, testers, product line managers, and sometimes members of the general public.

This paper describes the results to date in researching and prototyping a Next Generation Process Model (NGPM) and support system (NGPSS) which directly addresses these issues. The NGPM emphasizes collaborative processes, involving all of the significant constituents with a stake in the software product. Its conceptual basis is a set of Theory W (win-win) extensions to the Spiral Model of software development.

Also, this sequential participation in stages of the software process leads to win-lose situations among the various stakeholders. These may result from coalitions of some parties at the expense of absent parties; from an "out of sight, out of mind" phenomenon; or from plain ignorance of absent parties' likely win conditions.

Table 1. Frequent Software Development Win-Lose Patterns (which generally evolve into lose-lose situations).

Proposed Solution	"Winners"	Loser
Quick, cheap, sloppy product	Developer, Customer	User
Lots of "bells and whistles"	Developer, User	Customer
Driving too hard a bargain	Customer, User	Developer

1. Introduction

1.1 Context and Motivation

Current processes and support systems for software requirements often reinforce a sequential consideration of the concerns of the various stakeholders in the software process and its resulting products. In the most frequent sequence, fairly vague expressions of user's information processing needs are translated by system analysts into a set of product specifications (sometimes very precise, sometimes fairly general, but often inaccurate in important respects).

These specifications are used by a customer to define a contract with a developer generally with minimal user participation. Even if the first product of this contract is an initial increment of an evolutionary development, the detailed concerns of the product's users, maintainers, and interfacers are generally not explored until the initial increment is in operation. At that point, the product's architectural commitments often make it very difficult to reorient the product toward the user's, maintainer's, and interfacers' discovered win conditions.

Three of the most frequent win-lose situations are shown in Table 1. Building a quick and sloppy product may be a low-cost, near-term "win" for the software developer and customer, but it will be a lose for the user (and the maintainer). Adding lots of marginally useful "bells and whistles" to a software product on a cost-plus contract may be a win for the developer and users, but is a lose for the customer. And "best and final offer" bidding wars imposed on competing developers by customers and users generally lead to low-ball winning bids which place the selected developer in a losing position.

Actually, nobody wins in the above situations. Quick and sloppy products destroy a developer's reputation and have to be redone, inevitably at a higher cost to the customer. The "bells and whistles" either disappear or (worse) crowd out more essential product capabilities as the customer's budgets are exhausted. Inadequate low-ball bids translate into inadequate products, which again incur increased customer costs and user delivery delays to reach adequacy.

In order to break out of these losing software

patterns, we need software process models which explicitly emphasize continuous collaborative involvement of a software product's stakeholders in its definition and development stages. This paper describes a candidate for such a next-generation process model (NGPM), which has as its conceptual foundation, a set of Theory W (win-win) [Boehm-Ross, 1989] extensions to the Spiral Model [Boehm, 1988] of software development.

1.2 Overview of Content

Section 2 discusses how the elements of the Spiral Model and Theory W are integrated to produce the NGPM and describes the NGPM/NGPSS initialization process. Section 3 then elaborates primarily on the new sectors of the spiral model focused on Theory W steps: identifying software life-cycle constituents, identifying the constituents' win conditions, and reconciling the win conditions in order to determine the next level of project objectives, constraints, and alternatives. It shows examples of candidate Next Generation Process Support System (NGPSS) screens to illustrate how the NGPSS experimentation attempts to establish automated collaborative support for the NGPM. Section 4 indicates how the resulting objectives, constraints, and alternatives can be translated into an expanding object base of software specifications. Section 5 discusses related research. Some conclusions particularly relevant to software requirements are presented in Section 6.

2. NGPM and NGPSS Overview

2.1 Integrating Theory W and the Spiral Model

Figure 1 illustrates the Theory W extensions to the Spiral Model that form the conceptual basis for the NGPM. The two additional sectors in each spiral cycle, "Identify Next-Level Constituents" and "Identify Constituents' Win Conditions," and the "Reconcile Win Conditions" portion of the third sector, provide the collaborative foundation for the model. They also fill a missing portion of the current Spiral Model: the means to answer, "Where do the next-level objectives and constraints come from, and how do you know they're the right ones?" The refined Spiral Model also explicitly addresses the need for concurrent analysis, risk resolution, definition, and elaboration of both the software product and the software process, based on the Spiral Model extensions described in [Boehm-Belz, 1988].

In particular, the nine-step Theory W process in [Boehm-Ross, 1989] translates into the following Spiral Model extensions:

- **Determine Objectives.** Identify the system life-cycle constituents and their win conditions. Establish initial system boundaries, external interfaces.

- **Determine Constraints.** Determine the conditions under which the system would produce win-lose or lose-lose outcomes for some constituencies.
- **Identify and Evaluate Alternatives.** Solicit suggestions from constituents. Evaluate them with respect to constituents' win conditions. Synthesize and negotiate candidate win-win alternatives. Analyze, assess, and resolve win-lose or lose-lose risks.
- **Record Commitments,** and areas to be left flexible, in the project's design record and life cycle plans.
- **Cycle Through the Spiral.** Elaborate win conditions, screen alternatives, resolve risks, accumulate appropriate commitments, and develop and execute downstream plans.

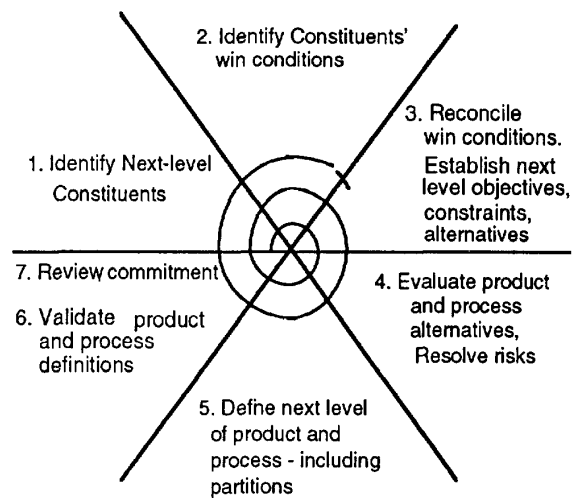


Figure 1. Theory W Extensions to Spiral Model

2.2 NGPM and NGPSS Overview: Initialization

Figure 2 show first-cut definitions of the NGPM initialization process used to get the first cycle of the NGPM spiral off to the right start. The formalism used here involves Perceptronics' CACE/PM® Tasknets, a variant of Petri Nets [Madni, 1988]. We have also experimented with an IDEF representation to define these processes and have found that a representation supporting both the process and product views is preferable for NGPM/NGPSS. We also plan to experiment with Arcadia's APPL/A [Osterweil, 1987] and the emerging Prototech/USC-ISI Process Virtual Machine concepts [Balzer, 1993], which have recently been coordinated with Scacchi's Articulator approach [Mi-Scacchi, 1992].

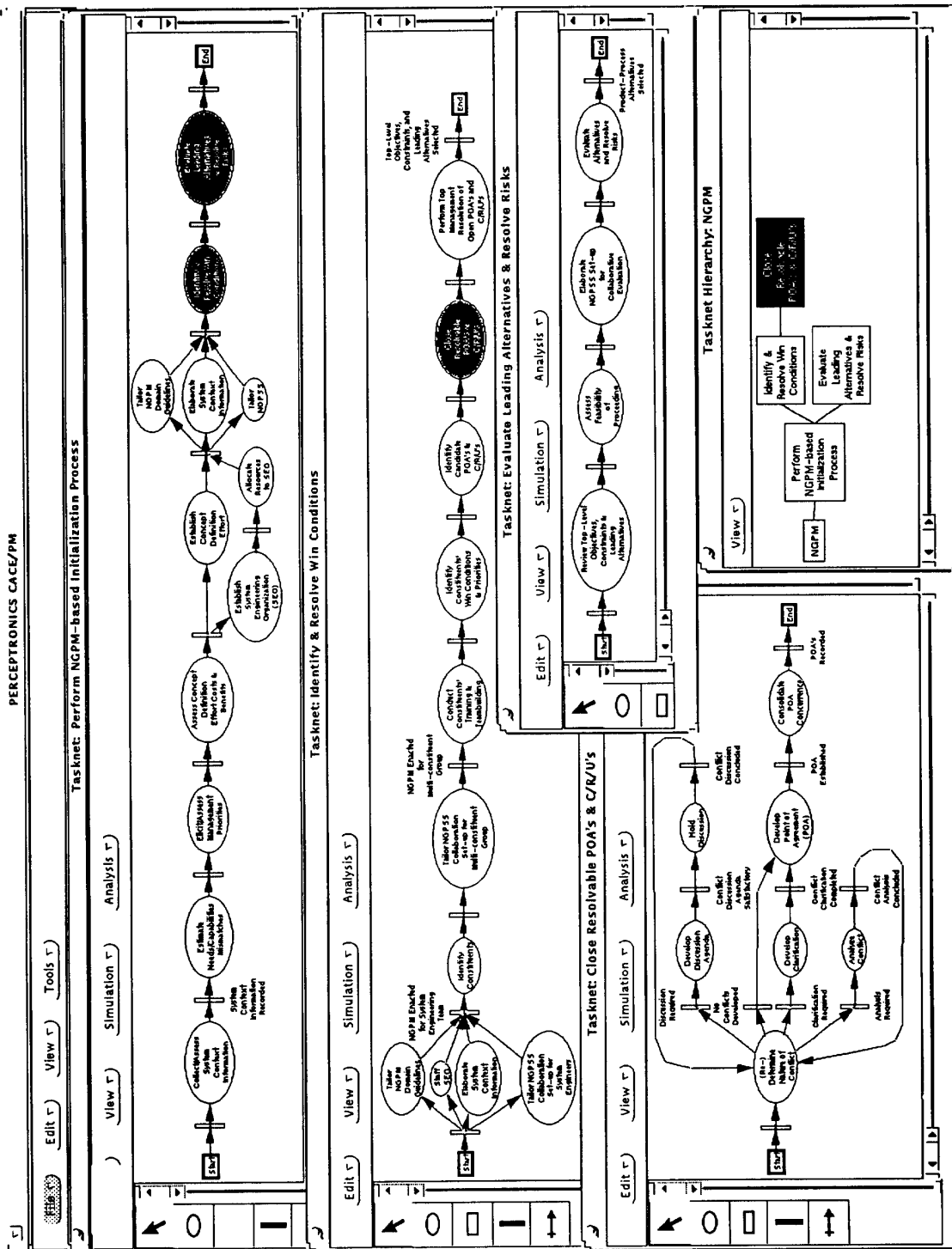


Figure 2. NGPM Initialization Process

The "Tasknet: Perform NGPM-based Initialization Process" in Figure 2 provides the overall context for applying the NGPM. It indicates that the primary entry condition for establishing an NGPM effort is an organizational assessment of information processing needs/capabilities mismatches, with a sufficient expected positive balance of benefits vs. costs in a particular need area to support establishing a Systems Engineering Organization (SEO) to conduct an initial cycle of the NGPM spiral. The SEO for the project is convened to ensure that the NGPM steps are appropriately prepared for, coordinated, and analyzed.

The "Tasknet: Identify and Resolve Win Conditions" in Figure 2 shows the primary process for navigating through the Theory W sectors 1, 2, and 3 of the NGPM spiral. Note that the Tasknets serve to guide the project through the appropriate sequences of steps (e.g., domain-tailoring of the NGPM, staffing the SEO, elaborating system context information, and tailoring the NGPSS collaboration setup), and to provide a means for monitoring progress. Note also that the processes are tailorable within the CACE/PM framework, so that the project's version of the NGPM can be matched to the project's priorities, domain characteristics, and boundary conditions -- both at the beginning of the project and during the project's evolution. The other tasknets shown in Figure 2 are described in later sections of this paper.

Further elaboration of the "Tasknet: Identify and Resolve Win Conditions" is shown in Figure 3. The figure shows how the step "Tailor NGPSS Collaboration Setup for Multi-Constituent Group" can be elaborated into a series of Resource Hierarchies for the classes of participating life-cycle constituents and their attributes; and for the support tools to be used in the collaboration process, such as the Constructive Cost Model (COCOMO) for cost/schedule/performance/functionality negotiation support, and b) an associated Activity Frame for monitoring progress on the setup task itself. Here again, the hierarchies and frames are tailorable within the CACE/PM framework to the project's special concerns.

3. The NGPM Spiral Sectors and their NGPSS Support

The spiral sectors of the NGPM, and the associated NGPSS support, guide the project through the cycles of increasing project definition, culminating in the system's Initial Operational Capability and embarkation into a set of spirals guiding the system's subsequent evolution. Progress through the sectors is predominantly clockwise, but not necessarily purely sequential. Here again, the tailorable but formalized structure of the NGPM and NGPSS allows for variations in task sequencing without losing controllability.

3.1 Sector 1: Identify Next-Level Constituents

The initialization of this sector was described above with respect to the Constituents Resource Hierarchy in Figure 3, which shows a default set of candidate constituents: interfacers, maintainers, customers, users, user representatives, user managers, operators, and developers. In the initial cycle of the spiral, the constituents would primarily be organizational representatives. In subsequent cycles, the constituents would increasingly involve individuals' concerns: individual operators at workstations or vehicle controls; individual liaison functions between interfacing organizations and systems; individual experts defining critical system capabilities.

The framework allows extension at any point to include additional categories of constituents. A Product Line Manager constituent, if not already accommodated as a Customer constituent, would have win conditions ensuring that reuse and reengineering would be emphasized in the life-cycle solution. Also, an attractive approach for ensuring the social, safety, or environmental acceptability of an information system is to include an appropriately representative "general public" constituent with win conditions to be addressed by the system.

3.2 Sector 2: Identify Constituents' Win Conditions

The NGPM and NGPSS provides experience-based, domain-specific prompts, defaults, and suggestions to aid constituents in establishing a full set of win conditions. For example, Figure 4 shows a NGPSS screen format for identifying interfacers function win conditions associated with a hypothetical Joint Task Force Command and Control System (JTFCCS) system definition, where the interfacers constituent represents the Army WWMCCS Information System (AWIS). The Command, Control, and Communications (C3) domain is used here as a representative large-scale, complex applications domain.

Once the Interfacers Function option is selected within the C3 domain, a win condition window opens up with prompts reflecting the classes of win-condition functions most likely to be of interest to a system such as AWIS, which will be interfacing with a new C3 system under development such as JTFCCS. These include communicating status-of-forces information from the new system to the interfacers, providing information to the interfacers on changes to the new system, etc. The prompts are the phrases before the ":" symbols in the window; the information after the ":" symbols would be entered by the AWIS interfacers representative.

Such domain-specific and constituent-specific prompts can be initialized by experience with previous C3 interface concerns, and refined by analysis of subsequent usage of

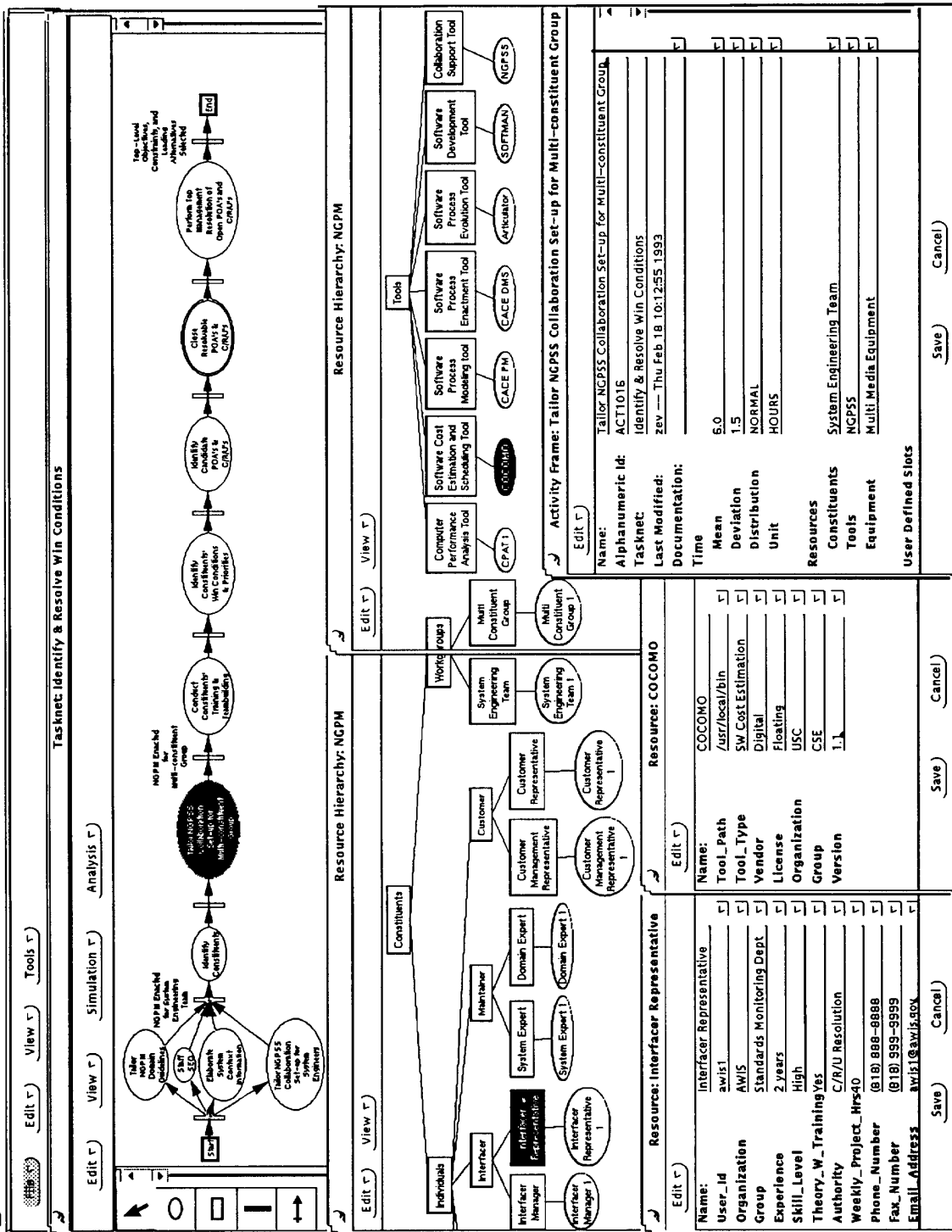


Figure 3. Example Initial NGPSS Screen Format

the NGPM. This particular approach combines the power of a domain-specific approach with the scalability and genericity of the win condition window approach. The NGPM research project is developing and experimenting with initial prototype sets of prompts, defaults, and suggestions in two domains, to test the genericity of their representations and operation. One domain is software engineering environments (SEE's), for which the authors have supplied a great deal of previously compiled win condition information, which would also be of considerable value to other SEE research.

We are also conducting research and experimentation on a number of issues critical to the development of efficient and effective means of capturing constituents' win conditions. These include determination of appropriate levels of detail for the prompts to be supplied in successively detailed spiral cycles; methods for characterizing and determining relative priorities of win conditions (e.g., "critical, important, desirable, optional"); collaboration semantics issues (e.g., avoiding terms with non-negotiable connotations such as "requirements" and "necessary"); and the expression and handling of near-term vs. long-term or deferrable win conditions. Another research topic involves investigating the most effective user interface styles among such alternatives as the hierarchy-and-frame organization of Figures 2 and 3, and the menu-list-frame orientation of Figures 4, 5, and 6. These differences are shown here as an indicator of the need for such research and experimentation.

3.3 Sector 3: Reconcile Win Conditions; Establish Next-Level Objectives, Constraints, Alternatives

Negotiation and reconciliation of win conditions is a complex, people-intensive process, involving considerable iteration with the activities in the previous sector (identifying win conditions) and subsequent sector (evaluating alternatives and resolving risks). Following the stepwise Theory W approach in [Boehm-Ross, 1989], three major activities are involved:

- a. Establishing relationships among win conditions;
- b. Searching out win-win arrangements;
- c. Translating these into objectives, constraints, and preferred alternatives (with associated rationales) in the project's object base.

Each of these activities is elaborated below.

a. Establishing Relationships Among Win Conditions

To meet its scalability goals, the NGPM/NGPSS attempts not to overautomate the classification and organization of win conditions. Research into

sophisticated inferencing and constraint-maintenance capabilities have been deferred until a base of experience is developed on how these functions are performed by experienced system engineers. Instead, the initial NGPSS support has primarily been based on forms of experience-based prompts, defaults and suggestions, and interactive support for establishing relationships between constituents' win conditions.

The two primary support capabilities for win condition reconciliation, provided by the NGPSS are for resolving Points of Agreement (POA's) and Conflicts/Risks/Uncertainties (CRU's). POA's are constituent win conditions which have no apparent conflicts with other win conditions, but which need explicit concurrence by other constituents. CRU's are groups of win conditions for which some class of collaborative conflict, risk, or uncertainty resolution must be performed to convert the issue into a POA.

b. Searching out Win-Win Arrangements

Figure 5 provides an example of a POA frame. It indicates an Army WIS win condition (to be provided with change information on JTFCCS outages) with no apparent conflict, and with a recommended closure process developed by the system engineering organization. The closure process indicates that a positive concurrence should be signified by the JTFCCS constituent, and that the Air Force interfacier, AFCCS, be provided an opportunity to participate in the closure process. However, the AFCCS constituent can concur by simply taking no action by the suspense date of Jan. 25, 1995.

Figure 6 provides an example of a conflict to be resolved via the CRU frame. Here, the conflict involves different multi-level security (MLS) interface specifications identified as win conditions by the AWIS and AFCCS constituents, with no MLS interface specification identified to date by JTFCCS. The CRU frame provides various attribute and relation slots, and linkages to further closure-related information such as a CRU closure plan.

The "Tasknet: Close Resolvable POA's and CRU's" in Figure 2 provides a generic CRU closure plan. CRU's are divided into three operational categories:

- "Clarification required" indicates that conflict resolution may simply involve clarification of which MLS interface specification is the most current one.
- "Discussion required" indicates that a discussion among the constituent parties may be sufficient to resolve the conflict.

Figure 4. Example NGPSS Support for Win Condition Identification

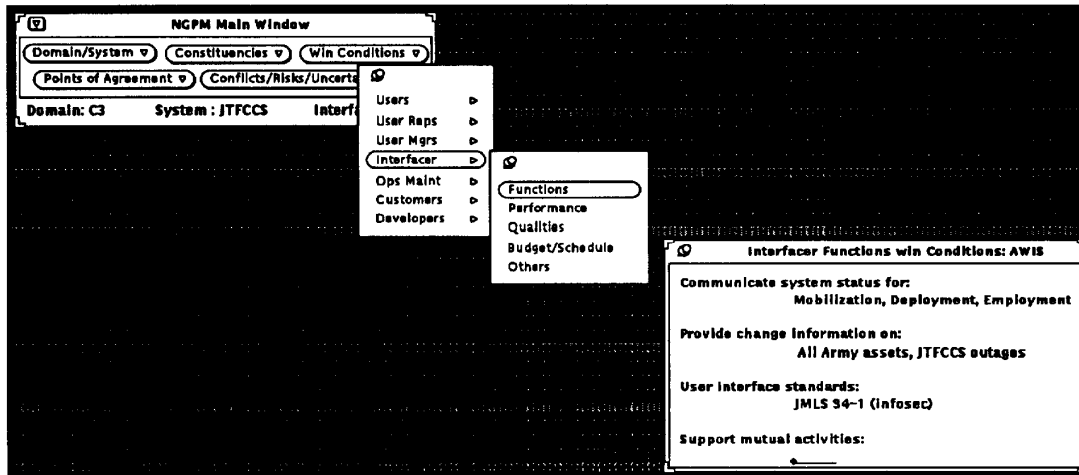


Figure 5. Support for points of Agreement (POA) Resolution

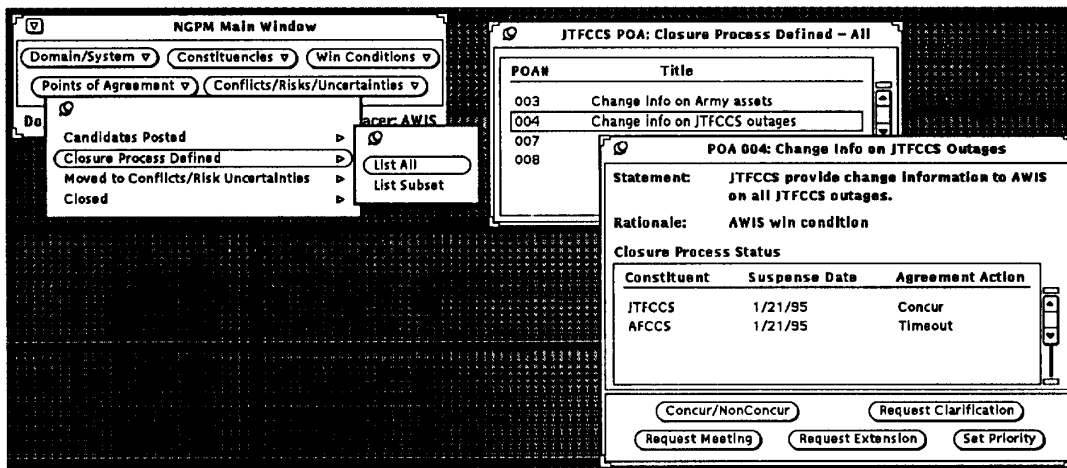
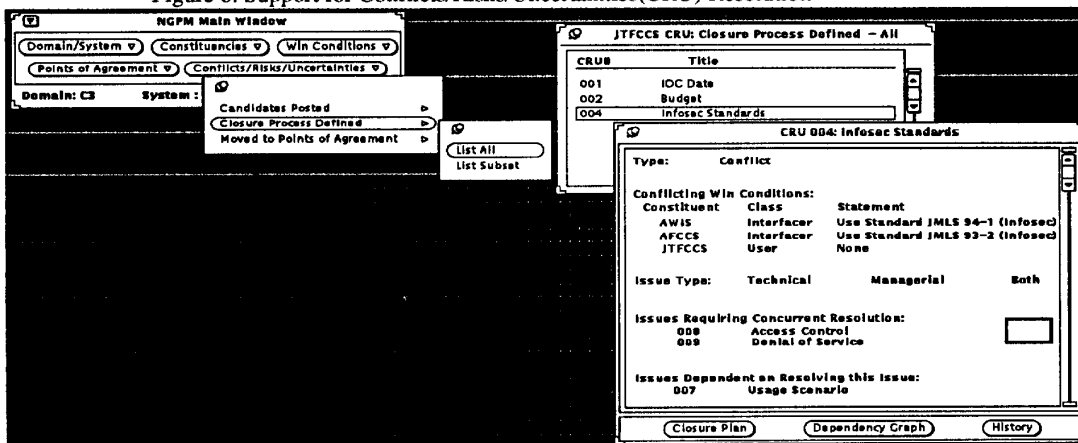


Figure 6. Support for Conflicts/Risks/Uncertainties(CRU) Resolution



- "Analysis required" indicates that substantive differences exist between the MLS specifications, which require some analysis to determine their effect on constituents' win conditions, and perhaps some creativity in searching out a win-win-win arrangement among the three constituents.

The Tasknet elaboration capability can support additional layers of definition of steps such as "Analyze Conflict" in Figure 2, and corresponding support for tracking progress and updating conflict analysis and resolution plans. These additional layers include the Theory W approaches in [Boehm-Ross, 1989] for searching out and establishing win-win arrangements, such as:

- Establishing reasonable expectations via teambuilding and objectives analysis.
- Expanding the option space by creating new alternatives.
- Time-sequencing of win conditions via incremental development.
- Refining and realigning win conditions.
- Matching people's tasks and incentives to their win conditions.

They can also draw on the Tools hierarchy in Figure 3 to support analysis and negotiation. Some additional candidate tools are a set of knowledge-based tools for cost/risk assessment, technical risk assessment, and process planning guidance currently being prototyped at USC.

Initial tool integration is loosely supported via wrappers. For example, a COCOMO wrapper can be defined by a file of COCOMO cost drivers as its input and a file of cost and schedule estimates as its output. In later stages, tool integration would be via a project object base: a software Work Breakdown Structure (WBS) with COCOMO cost drivers and cost/schedule estimates as WBS element attributes.

c. Translation into Object Base Objectives, Constraints, and Alternatives

The POA's resulting from the collaborative closure process form the basis of the growing object base of related objectives, constraints, and alternatives for the software project. The results of the clarifications, discussions, analyses, and risk resolution activities similarly form the basis of the rationale portion of a project's or a software component's "design record," associated with the software artifact to provide context for avoiding subsequent dysfunctional evolution decisions. The requirements aspects of this translation are summarized in section 4 below.

3.4 Sectors 4 through 7

Sectors 4 through 7 of the elaborated spiral model in Figure 1 are basically the same as sectors 2 through 4 and the Review/Commitment step in the original spiral model [Boehm, 1988], as extended to cover both product and process concerns in [Boehm-Belz, 1988]. Formalization and refinement of the process model and its steps will be done via process-PDL, process programming, and eventually process megaprogramming experiments. Further information on the NGPSS implementation is provided in [Boehm et. al., 1993]

4. NGPM and NGPSS Requirements Information Representation and Support

A major focus is on the elaboration of NGPM win conditions into an expanding sequence of object-base refinements. For example, the win condition, "Provide change information to AWIS on all Army assets" (POA #003 in Figure 5), can be represented by a method, "Provide change information to AWIS," applied to the subclass "Army assets" of asset objects within the JTFCCS object base. For example, as shown in Figure 7, the next spiral cycle distinguishes two subclasses (deployed and nondeployed Army assets), based on their need for different treatment via methods. Both the method and the subclass definitions are expanded into greater detail during subsequent spiral cycles.

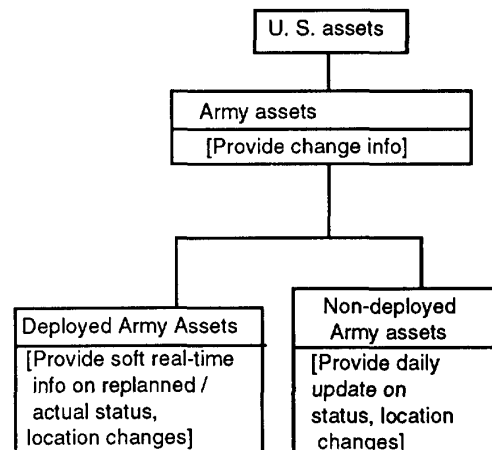


Figure 7. Translating Win Conditions into Object-Oriented Specs

Within this object-oriented specification context, experimentation with NGPM requirements and architecture specifications would address the three major deficiencies experienced with current software specifications such as those in DoD-STD-2167A:

- **Uniform precision.** There are no provisions to defer details in a requirements specification until options are better understood.
- **Uniform priorities.** There is no information support for making decisions on what to descope in a design-to-cost or reduced-budget situation, or decisions on which capabilities to defer in an incremental development.
- **Single-instant snapshots.** There is no information support for designing a software life-cycle architecture around likely sources of growth and change.

The uniform-precision deficiency can be addressed via the Spiral Model concept of risk-driven specifications. These enable software specifications to be rigorous where they need to be rigorous (e.g., interface specs to tightly-coupled external systems), and flexible where they need to be flexible (e.g., the look and feel of a user interface). The expanding object-oriented approach described above is well suited to risk-driven selection of which classes, objects, and methods to expand at a given point in time.

The uniform-priority deficiency can be addressed via the win condition priority attributes (critical, important, desirable, optional) and the similar deferability attributes discussed in Section 3. These provide the necessary information to connect constituents' win condition priorities to the project's determination of life cycle strategies and increments.

The single-instant snapshot deficiency can be addressed by having additional win condition and specification elements representing directions of life-cycle growth and change. These can be reinforced in the NGPSS by domain-specific prompts, defaults, and suggestions, both for the growth and change specifications, and for architectural strategies to accommodate them. For example, the growth source "workload volume" could be expressed in terms of number of threat objects, sensor data rates, transaction rates, or number of active users, depending on the domain. Counterpart architectural strategies for accommodating growth in workload volume could involve a mix of initially overbuying on capacity, providing balanced upgrade paths, and using open interface specifications to accommodate higher-performance equipment at a later time.

5. Related Research

The NGPSS approach to coordination support falls into the category of structured language-action systems exemplified by Flores and Winograd's Coordinator [Flores et al., 1988]. The initial NGPSS approach is aimed at providing more structure than the Coordinator, but not as

much as attempted in gIBIS [Conklin-Begeman, 1988], SIBYL [Lee, 1990], and REMAP [Ramesh-Dhar, 1992], which have difficulties in scaling up to large systems. The NGPSS degree of automation is roughly similar to that of Object Lens [Lai et al., 1988] which brought attention to task coordination through rule-based processing of structured e-mail messages; ISHYS [Garg-Scacchi, 1989] which tied rule-based hypertext and message management to software process support; on experience-based domain-oriented prompts, defaults, and suggestions; and on the conceptual bases for collaboration and software/system development provided by Theory W and the Spiral Model.

Additional related work involves the requirements engineering based on the representation of multiple viewpoints [Finkelstein et al., 1989; Easterbrook 1991; Dardenne 1991; Feather 1989]. Specifically the interactive approach to conflict exploration and resolution in Synoptic support system [Easterbrook 1991] has similarities to interactive searching of win-win arrangements leading to generation of POAs and CRUs. One of the limitations of Synoptic that NGPSS tries to address is multi-party involvement in the process - Synoptic allows only two viewpoints to be compared at once. The other major differences arise from the use of Theory W as a basis for collaborative requirements generation, use of domain specific representations and default domain specific knowledge to guide win condition elicitation and focus exploration of win-win solutions via POAs and CRUs. Apart from domain specific knowledge NGPSS also supports use of domain independent analysis tools like COCOMO, etc., that can be used as tools for evaluating win conditions and determining CRUs.

6. Conclusions

In this paper, we described the results to date in researching and prototyping a Next Generation Process Model (NGPM) and support system (NGPSS) which directly address the issues described in section 1.0. The NGPM emphasizes collaborative processes, involving all of the significant constituents with a stake in the software product. Its conceptual basis is a set of Theory W (win-win) extensions to the Spiral Model of software development.

We described the Next-Generation Process Support System (NGPSS) which is a groupware-oriented support capability for the NGPM. NGPSS enables an approach for collaborative win-condition elicitation and resolution with respect to the prospective software product's constituents or stakeholders. This approach is based on the Theory W steps of win-condition identification; expectations management; collaborative creation, analysis, and negotiation of win-win solutions; and management of win-lose or lose-lose risks. The NGPSS is elaborated into

a candidate set of user interface screens and collaborative support capabilities, based on Perceptronics' Computer Aided Concurrent Engineering (CACE/PM®²) toolset. The paper also elaborates on the translation of constituents' win conditions into a set of Spiral Model risk-driven, object-oriented requirements specification. These specifications emphasize several important properties frequently missing from requirements specifications: variable precision, priorities, and directions of life-cycle change. In our current work, we have been experimenting with win conditions, POAs and CRUs that pertain to the current version of NGPSS itself. The objective is to better understand the strengths and weaknesses of the Theory W based process model of requirements elicitation that satisfy multiple constituents and to simultaneously extend NGPSS to provide the necessary computational support for such processes.

7. References

- [Balzer, 1993]. R.M. Balzer, "Language Independent Generic Process Evolution," Proceedings, International Workshop on Evolution of Software Processes, Montreal, January 1993.
- [Boehm, 1988]. B.W. Boehm, "A Spiral Model of Software Development and Enhancement," Computer, May 1988, pp. 61-72.
- [Boehm-Belz, 1988]. B.W. Boehm and F.C. Belz, "Applying Process Programming to the Spiral Model: Lessons Learned," Proceedings, IEEE 4th Software Process Workshop, May 1988.
- [Boehm-Ross, 1989]. B.W. Boehm and R. Ross, "Theory W Software Project Management: Principles and Examples," IEEE Trans. Software Engr., July 1989.
- [Boehm et. al., 1993]. B. W. Boehm, P. Bose, E. Horowitz, W. Scacchi, S. Bendifallah and A. Madni, "Next-Generation Software Processes and Their Environment Support", USC Center for Software Engineering Report, June 1993.
- [Conklin-Begeman, 1988]. J. Conklin and M. Begeman, "gIBIS: A Hypertext Tool for Exploratory Policy Discussion," ACM Trans. Office Info. Systems, Oct. 1988, pp. 303-331.
- [Dardenne, 1991]. A. Dardenne, S. Fickas and A. Lamsweerde, "Goal-directed Concept Acquisition in Requirement Elicitation", Proceedings, Sixth International Workshop on Software Specification and Design, Oct. 1991, pp. 14-21.
- [Easterbrook, 1991]. S. Easterbrook, "Handling Conflict between domain descriptions with Computer-supported Negotiation", Knowledge Acquisition, Vol. 3, 1991, pp 255-289.
- [Feather, 1989]. M. S. Feather, "Constructing specifications by combing parallel elaborations", IEEE Trans. on Software Engr. 15, 1989, pp 198-208.
- [Finkelstein et. al., 1989]. A. C. W. Finkelstein, M. Goedicke, J. Kramer, and C. Niskier, "Viewpoint oriented software development: methods and viewpoints in requirements engineering", Proceedings of the 2nd Meteor Workshop on Methods for Formal Specification, Springer, 1989.
- [Flores et. al., 1988]. F. Flores, M. Graves, B. Hartfield, and T. Winograd, "Computer Systems and the Design of Organizational Interaction," ACM Trans. Office Info. Systems, April 1988, pp. 153-172.
- [Garg-Scacchi, 1989]. P. K. Garg and W. Scacchi, "ISHYS: Designing an Intelligent Software Hypertext System", IEEE Expert, Vol. 4(3), Fall 1989, pp 52-62.
- [Lai et al., 1988]. K-Y. Lai, T.W. Malone, and K-C Yu, "Object Lens: A "Spreadsheet" for Cooperative Work," ACM Trans. Office Info. Systems, October 1988, pp. 332-353.
- [Lee, 1990]. J. Lee, "SIBYL: A Qualitative Decision Management System," in Artificial Intelligence at MIT: Expanding Frontiers, P. Winston and S.Shellard, eds., MIT Press, 1990, pp. 106-133.
- [Madni, 1988] A.M. Madni, "HUMANE: A Designer's Assistant for Modeling and Evaluating Function Allocation Options," Proc. Ergonomics of Advanced Manufacturing and Automated Systems Conf., Louisville, KY(August 1988)
- [Mi-Scacchi, 1992] P. Mi and W. Scacchi, "Process Integration for CASE", IEEE Software, Vol. 9(2), 45-53, (March 1992). Also appears in Computer-Aided Software Engineering, (2nd. Edition), E. Chikofski (ed.), IEEE Computer Society (1993)
- [Osterweil, 1987]. L. Osterweil, " Software Processes Are Software Too," Proceedings, ICSE 9, March 1987, pp. 2-13.
- [Ramesh-Dhar, 1992]. B. Ramesh and V. Dhar, "Supporting Systems Development by Capturing Deliberations During Requirements Engineering," IEEE Trans. SW Engr., June 1992, pp. 498-510.

² CACE® by Perceptronics